Projet IA & Application

Souhila Arib

CY-Tech ING3 IAC Semaine A

Groupe - ANDRÉ Victor, BOGAER Youenn, DANO Ewen

03/05/2026

Sommaire :

CY-Tech ING3 IAC Semaine A.... 1
Groupe - ANDRÉ Victor, BOGAER Youenn, DANO Ewen.... 1
03/05/2026.... 1
Sommaire :.... 2
Introduction.... 3
Contexte.... 3
Pourquoi ce projet ?.... 3
Données.... 4
Features:.... 4
Target :.... 6
Classification :.... 6
Régression :.... 6
Implémentation.... 6
Kedro.... 6
MLFlow.... 8
Optuna.... 8
Modèle : Xgboost.... 9
Qu'est-ce que XGBoost ?.... 9
Pourquoi XGBoost est-il plus performant qu'une Random Forest ?.... 9
Pourquoi l'avoir choisi pour le S&P 500 ?.... 9
Les forces de l'algorithme.... 10
Notre implémentation de XGBoost :.... 10
Classification :.... 10
Régression :.... 10
La recherche de seuils.... 10
Tests :.... 11
Métriques :.... 11
Métriques de classification :.... 11
Métrique de régression :.... 12
Métriques de Trading (Performance Financière).... 12
Outils de Visualisation.... 12
Analyse.... 12
Conclusion.... 15
Limites de notre projet.... 15
Bonus :.... 15

Code source :
Vous pouvez retrouver l'entièreté du code source sur le repository suivant :

https://github.com/V-Y-E-CYTech2026/ia_app

Introduction

Ce projet se base sur une compétition de machine learning Kaggle dont le but était de battre un indice boursier sur une longue période.
A savoir que les inscriptions au concours de Kaggle était déjà fermées, nous n'avons pas pu nous inscrire afin de présenter notre projet

lien kaggle : https://www.kaggle.com/competitions/hull-tactical-market-prediction/overview

Contexte

L'indice boursier en question est le s&p500 qui représente les 500 plus grandes entreprises américaines. Ce classement n'est pas fixe, en cas de mauvais résultats, une entreprise peut être remplacée par une autre plus performante. Le s&p500 représente 80% de la capitalisation boursière des États-Unis, soit plus de 61 000 milliards d'USD au 31 Décembre 2025.
Ce projet était présenté par Kaggle comme assez difficile car le s&p500 est très volatile, il est très dépendant des événements extérieurs. De plus comme vu, cet indice garde uniquement les entreprises cotées les plus performantes. De ce fait, le s&p500 ne fait que monter, dû à la croissance économique constante sur le long terme. Nous verrons que cela peut poser problème.

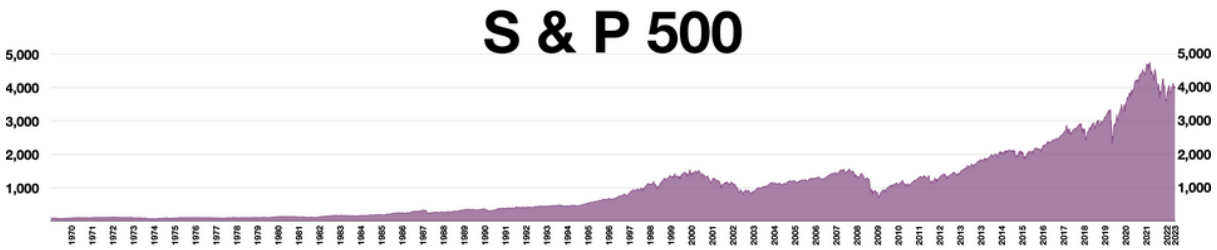

Figure 1 : Evolution du S&P 500 entre 1970 et 2023

Pourquoi ce projet ?

Nous avons choisi ce projet car il s'éloigne des projets académiques que l'on a pu réaliser cette année. Les tâches que nous avons réalisées vont au-delà de la simple implémentation d'un modèle, nous en avons profité pour appliquer les notions du cours de MLops, jusqu'à arriver à un quasi-produit fini (full-stack). Par ailleurs, nous avons implémenté beaucoup de modèles de deep learning cette année, mais peu de machine learning. Nous en avons profité pour implémenter un vrai modèle de Machine Learning.

Données

Nous ne nous sommes pas inscrits à la compétition Kaggle liée au projet, de ce fait nous n'avions pas accès aux données initialement prévues pour le concours.
Nous avons cherché des données. Cette partie a été l'une des plus chronophages pour différentes raisons :

- Nous n'avions jamais étudié de près ou de loin la finance donc pour chaque feature proposée par d'autres modèles, nous devions trouver des définitions et essayer de comprendre ce qu'elle représentait réellement.
- Nous devions faire des tests pour voir si certaines features n'étaient pas trop corrélées pour ne pas biaiser l'importance des features dans notre modèle.
- Nous avons testé différentes combinaisons de features sur plusieurs entraînements pour chercher les plus pertinentes pour les performances de notre modèles

Nous avons pris toutes nos données entre 2000 et aujourd'hui, ce qui représente 6 636 lignes.

Nous ne pouvons pas garder toutes les features car notre modèle les aurait pris en compte même si leur importance était moindre donc cela aurait créé du bruit.

Certaines de nos features se concentrent sur le long terme (unemployment…) et d'autres sur le court terme (log_volume…).

La plupart des features sont récupérés en temps réel grâce au framework de la fed (federal reserve) disponible sur python.

Importance du data leakage :

Dans un modèle boursier il est très important d'être sûr que lors de l'entraînement ou des tests, le modèle n'ai pas accès à des données du futur, par exemple si on ne fait rien et qu'on entraîne le modèle avec les données du jour même alors à l'ouverture de la bourse le modèle connaît déjà toutes les données du jour même la ou les financiers non, donc notre modèle triche. C'est ce qu'on appelle du data leakage.
Pour contrer cela, toute nos variables courtes terme ,comme la variation du volume de transaction, ont été shifté de 1.

Mais les variables avec un horizon long terme, comme le taux d'inflation ou de chômage, ne sont publiées qu'une fois pas mois donc ces variables ont été shifté de 30 jours.

Features:

Feature	Explication Simple	Horizon (Court/Long Terme)
RSI	Indique si le marché est suracheté ou survendu.	Court terme
ADX	Mesure la force d'une tendance (peu importe la direction).	Court / Moyen terme
Z_Score_10	Compare le prix actuel à sa moyenne sur 10 jours pour voir s'il y a un écart anormal.	Court terme
Volatility	Mesure l'agitation des prix (l'incertitude du marché).	Court terme
Dist_SMA_10	Écart entre le prix et la moyenne des 10 derniers jours.	Court terme
Dist_SMA_200	Écart entre le prix et la moyenne des 200 derniers jours (tendance de fond).	Long terme
Log_Vol_Change	Variation du volume de transactions (intensité de l'activité).	Court terme
Trend_Signal	Signal binaire ou numérique indiquant si la tendance est haussière ou baissière.	Moyen terme
Ret_L1 / L2	Rendements passés (L1 = hier, L2 = avant-hier).	Très court terme
VIX_Change	Variation de "l'indice de la peur". Anticipe la nervosité du marché.	Court terme
Fed_Rate	Taux d'intérêt de la Banque Centrale Américaine.	Long terme
Inflation_var	Changement du niveau des prix à la consommation.	Moyen / Long terme
stress_fed	Indice de stress financier calculé par la Réserve Fédérale.	Moyen terme
Yield_Spread	Différence entre les taux d'intérêt long terme et court terme.	Long terme
Yield_Spread_Ch.	Variation de cet écart (souvent utilisé pour prédire les récessions).	Moyen / Long terme

Nous voyons une grande différence d'horizon entre certaines de ces métriques, donc pour un entraînement avec un horizon faible (1-5 jours) nous ne garderons que les métriques avec un faible horizon, de même pour un entraînement à un horizon fort (>10j). Entre 5 et 10 jours, nous avons fait le choix de tout garder.

Target :

Classification :

Nous avons définis une première target de classification, notre but avec celle-ci est juste de prédire une montée ou une baisse du s&p 500, cette target sera notre principale car la plus simple à mettre en place.

Régression :

Nous avons aussi un target pour de la régression, avec celle-ci nous allons essayer de prédire de combien peut monter ou baisser le s&p500.

La prédiction d'un marché boursier peut-être très bruitée, une façon de réduire ce bruit est de ne pas faire des prédictions au jour le jour mais de le faire sur une plage de temps (5,10, 30 jours…), pour faire cela nous avons ajouté une logique dans notre code pour pouvoir simplement changer une variable dans kedro et générer des modèles avec un horizon plus élevé.

De plus, bien que nous ayons implémenté un modèle de régression nous allons principalement comparer nos résultats avec la target de classification. Car le s&p500 est tellement complexe, et les marchés boursier en général, que essayer de prédire précisément le taux de monté est très compliqué.

Implémentation

Initialement l'implémentation du projet s'est faite sur des notebooks jupyter. Cependant le cours de MLops nous a encouragé à utiliser Kedro et MLflow pour proposer un projet propre et conforme au bonne pratique en Machine Learning.

Kedro

Kedro est un framework Python open-source conçu pour faciliter et systématiser les pipelines de Data Science et Machine Learning en les rendant modulaires, robustes et reproductibles.
Parmi les apports de Kedro il y a le catalogue de données pour stocker les adresses dans un fichier yml, les Nodes qui représentent des tâches unitaires et les Pipelines qui sont des séries de Nodes.

En plus de rendre une pipeline complexe modulaire et lisible, grâce à un outil de visualisation des pipelines, Kedro permet facilement de collaborer avec d'autres applications de déploiement comme Airflow.

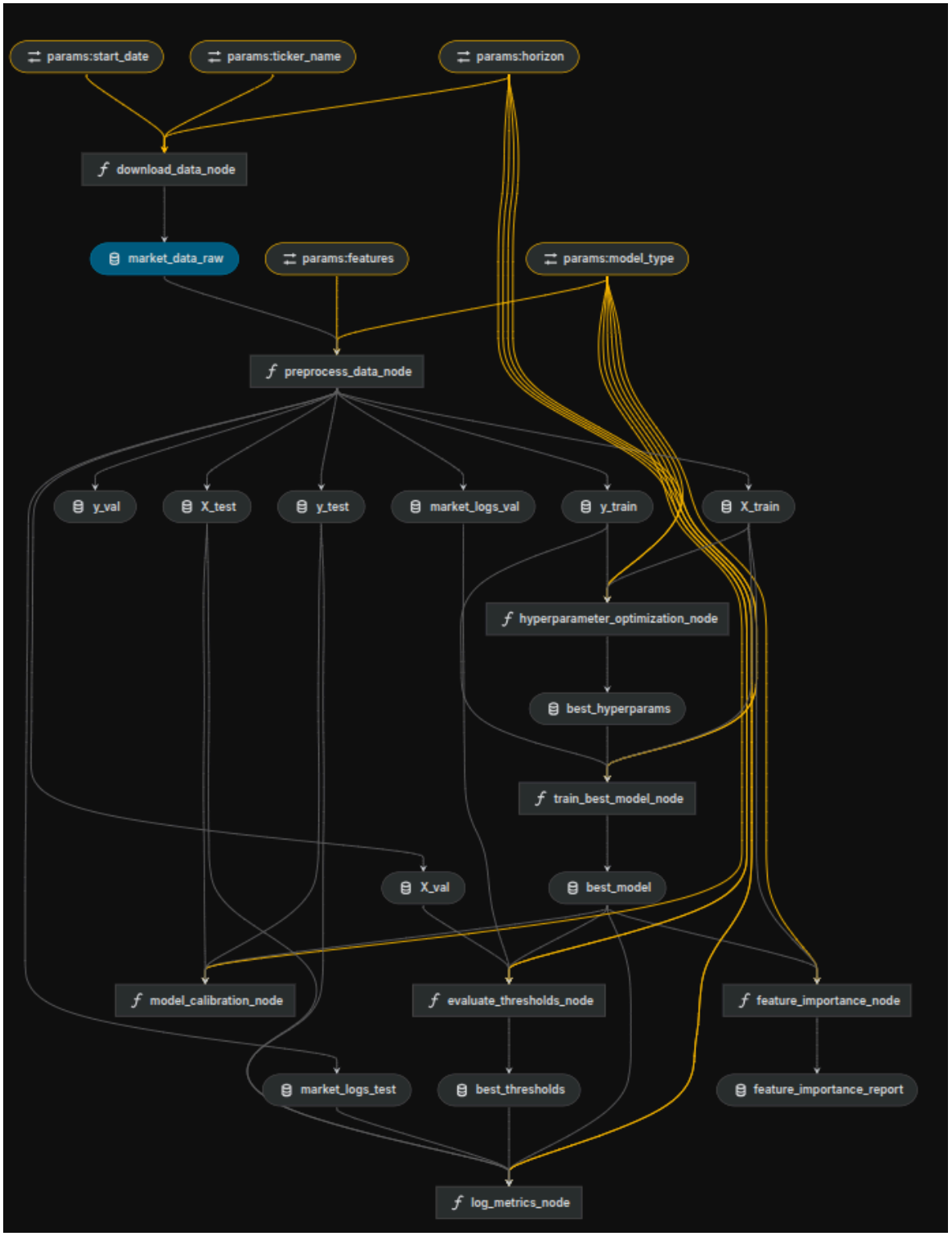

fig 1 : pipeline kedro

MLFlow

Mlflow est une plateforme open source agnostique (ne dépend pas des bibliothèques ou du langage) qui permet de gérer le cycle de vie d'un projet de Machine Learning.
Il intègre un tableau de bord contenant l'historique des paramètres d'entraînements et les résultats, les projets, les modèles et leurs versions.
Il intègre des outils de comparaisons entre les runs et la traçabilité. Et globalement c'est un outil de collaboration.
En pratique on appelle Mlflow dans les pipelines Kedro pour stocker les résultats d'entraînement.

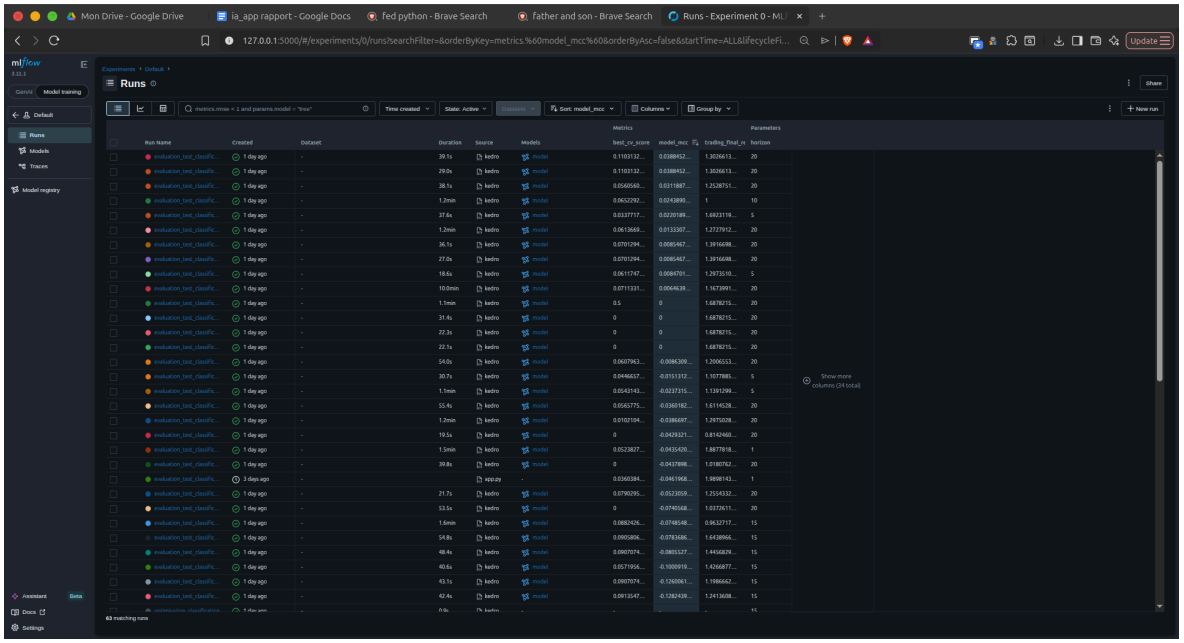

fig 2 : exemple interface mlflow

Optuna

Optuna permet de faire un grid search sur une plage de hyperparamètre donnée, cela est beaucoup plus efficace que réaliser un gridsearch brut.

Modèle : Xgboost

Nous avons commencé par un modèle simple, un arbre de décision. Cependant les résultats n'étaient pas très bons. Nous avons finalement mis en place un modèle XGBoost.

Qu'est-ce que XGBoost ?

XGBoost (eXtreme Gradient Boosting) est une implémentation optimisée de l'algorithme de Gradient Boosting. Contrairement à un arbre de décision classique qui essaie de prédire la cible en une seule fois, le boosting construit le modèle de manière séquentielle.

- Le principe : On entraîne un premier modèle simple (un "weak learner"). On calcule ses erreurs (les résidus). Ensuite, on entraîne un deuxième modèle pour corriger spécifiquement les erreurs du premier, et ainsi de suite.
- L'objectif : Minimiser une fonction de perte (loss function) en ajoutant des modèles qui "poussent" le résultat vers la valeur réelle.

Pourquoi XGBoost est-il plus performant qu'une Random Forest ?

Caractéristique	Random Forest	XGBoost
Méthode	Entraîne plusieurs arbres en parallèle et fait la moyenne.	Boosting : Entraîne les arbres en série ; chaque arbre corrige le précédent.
Gestion de l'erreur	Réduit principalement la variance (évite que le modèle soit trop instable).	Réduit le biais et la variance (le modèle est plus précis).
Optimisation	Simple forêt d'arbres.	Utilise la régularisation pour éviter le sur-apprentissage.

Pourquoi l'avoir choisi pour le S&P 500 ?

La Random Forest est excellente pour des données stables, mais elle a du mal à capter des tendances complexes ou des signaux faibles dans des données très bruitées comme la bourse. XGBoost, grâce à son système de correction d'erreur pas à pas, est beaucoup plus fin pour détecter ces micro-tendances. De plus, XGBoost gère nativement les données manquantes, ce qui arrive souvent avec des indicateurs financiers.

Les forces de l'algorithme

- La Régularisation : XGBoost possède des paramètres de pénalité (Alpha et Lambda). Cela empêche le modèle de devenir trop complexe, ce qui est crucial pour le S&P 500 afin d'éviter qu'il n'apprenne par cœur le bruit du marché.
- Gradient Descent : Il utilise l'optimisation par descente de gradient pour minimiser la perte, ce qui lui permet d'être extrêmement rapide et précis.
- Élagage en post-pruning : Contrairement aux arbres classiques qui s'arrêtent dès qu'ils rencontrent une difficulté, XGBoost construit l'arbre en entier puis le "taille" en remontant pour ne garder que les branches qui ont une réelle importance prédictive.

Notre implémentation de XGBoost :

Classification :

Le modèle va prédire que le s&p 500 monte ou non sur un horizon donné

Nous avons utilisé une calibration avec une méthode sigmoïd avec une validation croisée à 5 plis afin d'obtenir des probabilités fiables ce que Xgboost ne fait pas nativement, il est important pour nous d'avoir des probabilités fiables pour notre méthode de test.

De plus, comme le marché ne fait que monter, il y a un fort déséquilibre des classes alors pour compenser ce déséquilibre on applique un poids sur la classe minoritaire, ici le fait que le marché baisse.

Régression :

Ici nous cherchons à prédire la valeur numérique du rendement futur.
Il nous faut ici une fonction de perte et nous avons choisi le MSE (Mean Square Error)

$$L(y, \widehat{y}) = \sum (y_i - \widehat{y_i})^2$$

Avec la formule nous voyons que que l'on pénalise plus les erreurs sur les fortes baisses ou hausse (grâce à la mise au carré) qu'avec une fonction de perte tel que le MAE (mean absolute error).

Les rendements sont aussi très faibles, donc après nos premiers tests nous avons remarqué que ces hausses et baisses étaient négligeables par rapport au bruit, alors lors du prétraitement des données nous avons ajouté un facteur 1000 à la target d'entrée afin que notre modèle converge plus facilement et prenne le dessus par rapport aux bruits du marché

La recherche d'hyperparamètre :

Pour optimiser les performances de notre modèle, nous avons utilisé la bibliothèque Optuna en configurant un processus de recherche sur 400 essais. Ce volume important d'itérations était nécessaire pour explorer efficacement une plage d'hyperparamètres volontairement étendue. Nous avons employé l'algorithme TPE Sampler (Tree-structured Parzen Estimator), qui permet une recherche plus intelligente et rapide qu'un balayage aléatoire en apprenant des essais précédents.

Afin de respecter la chronologie des données financières et d'éviter toute fuite de données (look-ahead bias), l'évaluation de chaque combinaison s'est appuyée sur un TimeSeriesSplit à 5 plis. La métrique de validation retenue pour guider cette optimisation était le MCC.

Nos résultats indiquent que les meilleures performances sont obtenues avec un taux d'apprentissage (learning rate) faible et des profondeurs d'arbres limitées. Ces observations sont cohérentes avec la littérature spécialisée en finance, qui préconise des modèles peu complexes pour éviter que l'algorithme ne capture le "bruit" des données d'entraînement. Cette stratégie de simplification est complétée par des valeurs de reg_alpha et reg_lambda faibles, apportant une régularisation supplémentaire pour garantir une meilleure stabilité du modèle sur des données invisibles.

La recherche de seuils

Après notre entraînement, pour la classification, notre modèle peut juste nous dire si le marché va monter ou va baisser, en réalité dans ces probabilité, il y a une zone d’incertitude ou le modèle ne sait pas si le marché va monter ou baisser.
Donc il nous faut définir cette zone, pour se faire nous avons implémenté une fonction qui teste différents seuils des probabilités pour acheter ou shorter dans le marché. C’est comme une validation croisée sur les hyperparamètres mais avec des seuils de décisions.

Ces tests se font sur le dataset de validation.

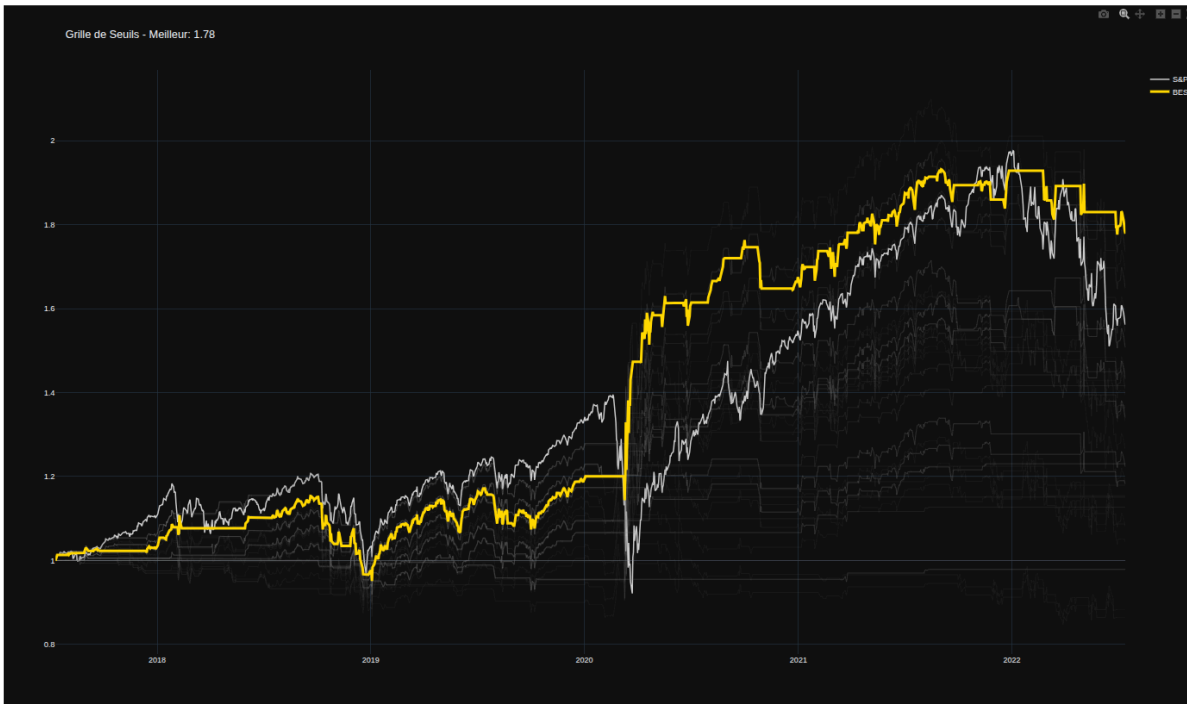

fig 3 : graphique de recherche des seuils

Tests :

Métriques :

Métriques de classification :

- **Accuracy**
- **Precision**
- **Recall**
- **F1-Score**
- **MCC (Matthews Correlation Coefficient) :** Très utilisé sur les marchés financiers, Extrêmement fiables pour les classes déséquilibrés car il utilise toutes les valeurs d'une matrice de confusion
  - $MCC = \frac{VP \times VN - FP \times FN}{\sqrt{(VP+FP)(VP+FN)(VN+FP)(VN+FN)}}$
- **ROC AUC :** Mesure la capacité du modèle à séparer les classes 0 et 1, peu importe le seuil choisi.
- **Log Loss :** Mesure la confiance du modèle . Plus le score est bas, plus les probabilités sont fiables.
  - $Log\ Loss = -\frac{1}{N} \sum_{i=1}^{N} (y_i \ln(p_i) + (1 - y_i) \ln(1 - p_i))$

Métrique de régression :

- MAE (Mean absolute error)
- MSE (Mean square error)
- R^2

Métriques de Trading (Performance Financière)

- Sharpe Ratio : Le rendement par unité de risque. Un score > 1 est bon, > 2 est excellent. Il punit la volatilité, même quand elle est positive.
- Sortino Ratio : Similaire au Sharpe, mais il ne punit que la volatilité négative (les baisses).
- Max Drawdown (Perte maximale) : La plus grosse chute d'un capital entre un sommet et un creux.
- Calmar Ratio : Rendement annuel divisé par le Max Drawdown. Il mesure le "rendement sur risque de catastrophe".
- Profit Factor : Le rapport entre la somme de tous les gains et la somme de toutes les pertes. Si > 1, la stratégie est rentable. Un score de 1.5 est souvent considéré comme très solide.
- Win Rate : Le pourcentage de jours où la stratégie a généré un profit positif lorsqu'elle était investie.

Outils de Visualisation

- Feature Importance
- Courbe de Calibration
- Backtest Curve

Analyse

Pour nos résultats nous avons choisi de tester notre modèle sur un horizon de 5 jours, cet horizon nous permet d'éviter le bruit et d'utiliser un maximum de variables, celles qui représentent le court terme et le long terme.

Le but premier était de battre le marché. Les chiffres montrent que l'objectif est atteint :

- Rendement final : Notre stratégie affiche un score de 1.69 (+69%) contre 1.56 (+56%) pour le S&P 500. Nous avons donc réussi à créer une "surperformance" (Alpha) de 13 points par rapport à l'indice de référence.
- Gestion du risque : Le ratio de Sharpe (1.06) et celui de Sortino (1.27) sont supérieurs à 1. Cela prouve que les gains ne sont pas dus au hasard, mais à une stratégie solide qui sait limiter les pertes lors des mauvaises périodes.

L'analyse des métriques de classification nous donne une vision précise du "caractère" de notre modèle :

- Le Rappel (Recall) de 85% : C'est le chiffre le plus marquant. Il signifie que le modèle est capable de détecter presque toutes les phases de montée du marché. Il préfère prendre le risque d'acheter "pour rien" plutôt que de rater une opportunité de gain.
- La Précision (60%) : Le modèle se trompe parfois, mais comme il capte l'essentiel des hausses, il reste largement rentable sur le long terme.
- Le MCC (0.02) : Bien que faible d'un point de vue purement statistique, ce score est classique en finance. Il montre que le marché est très bruité, mais que le modèle parvient tout de même à extraire une tendance payante.

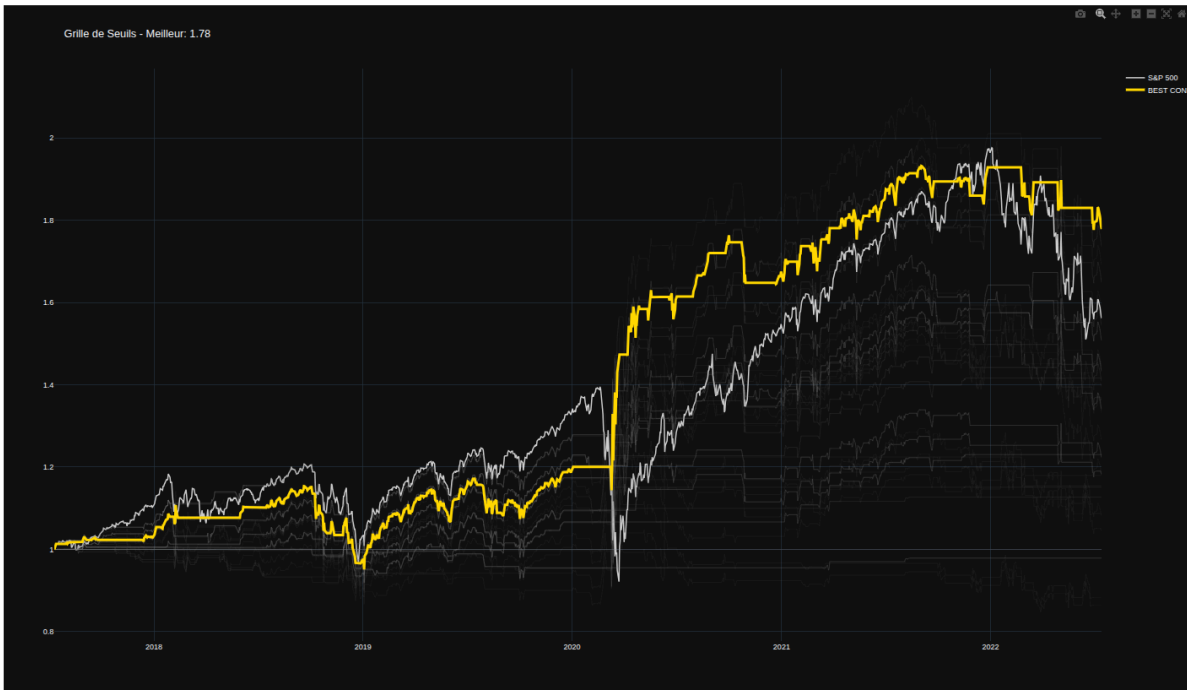

fig 4 : rendement du modèle

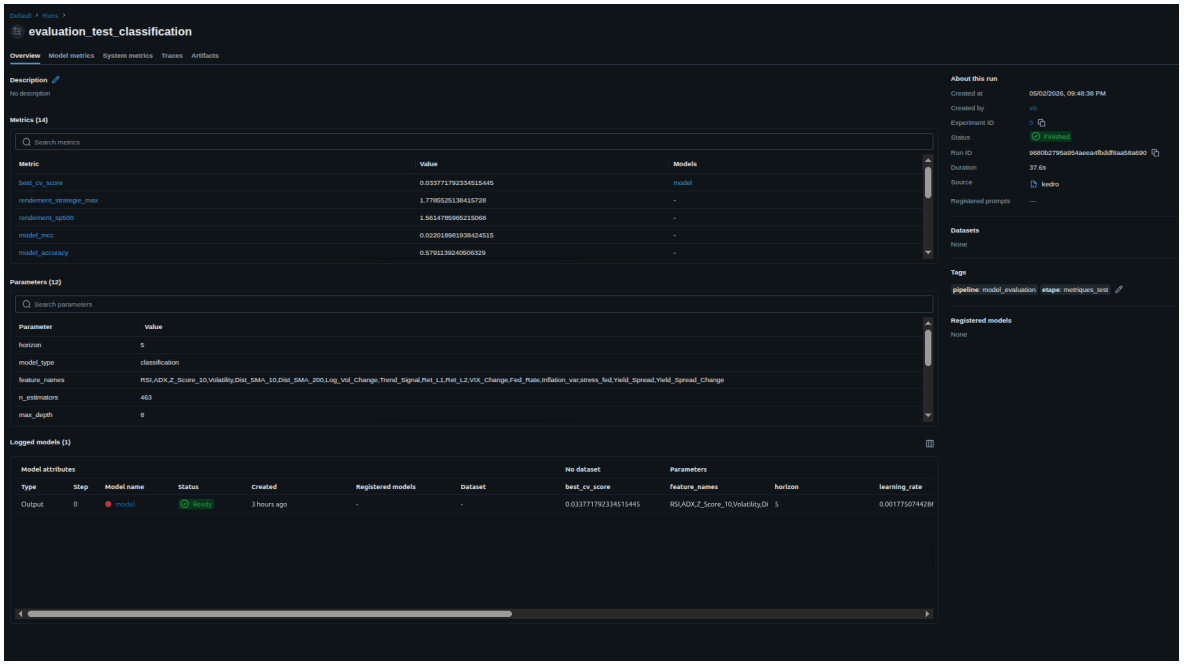

fig 5 : métriques du modèle

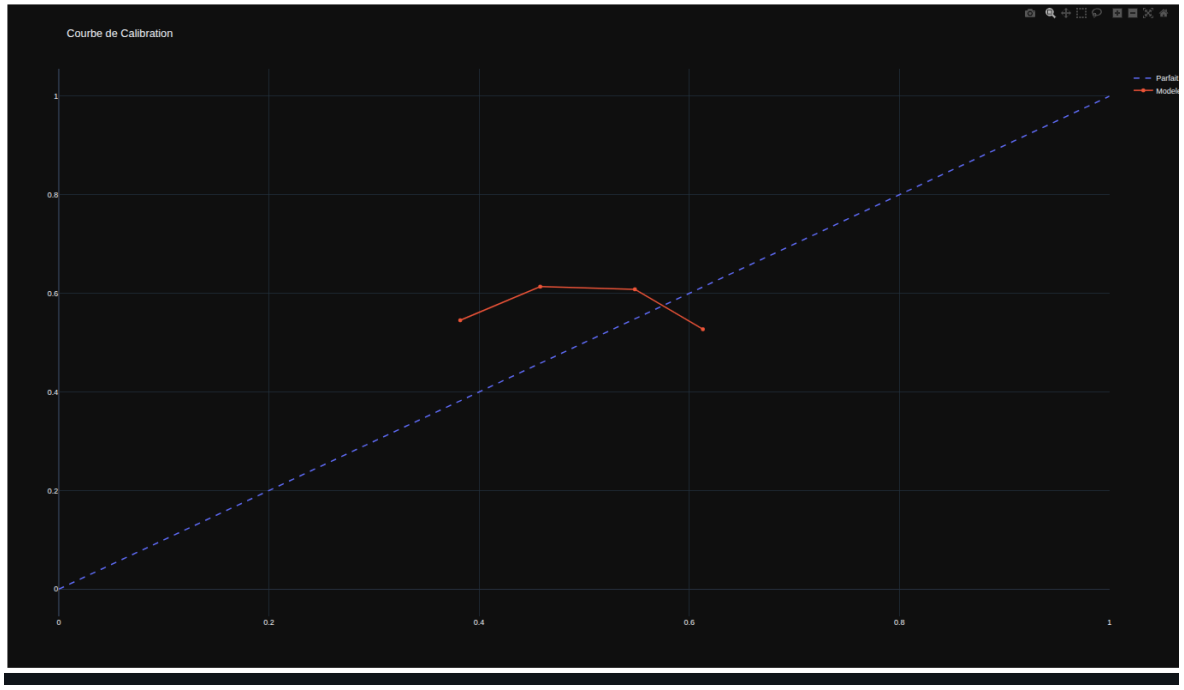

fig 6 : courbe de calibration

Conclusion

Limites de notre projet

Nous avons vu que nous avions eu un meilleur résultat que le s&p500, or nous avons choisi notre modèle, certains autres modèles qui présentaient de meilleurs métriques (le mcc par exemple) ne battait pas le s&p500 car ils pouvaient se tromper sur des jours avec une forte hausse ou une forte baisse. A l'inverse d'autres modèles qui avaient des moins bonnes métriques battent largement notre modèle, même certains modèles battaient de 1% le s&p500 cependant les métriques disaient que le modèle était moins bon que le hasard, mais le modèle a eu raison sur les jours les plus importants. Ces modèles étaient certes très performant sur l'intervalle de tests mais pas fiables

De plus, le modèle que nous avons choisi a été très performant sur notre période de test mais en l'essayant sur une autre période il est légèrement moins performant que le s&p500.

On ne doit pas oublier que même si ici notre modèle superforme de 13 points dans la réalité il y a des frais à chaque transactions donc nos bénéfices seraient beaucoup plus faibles.

Il est clairement impossible de pouvoir générer des modèles qui battent à coup sûr le s&p500, si c'était le cas tous les plus grands investisseurs les utiliseraient et dans tous les cas le marché monterait et il faudrait développer de nouveaux modèles.

Bonus :

Nous avons créé une api rapidement avec streamlit afin de pouvoir entraîner un nouveau modèle et d'obtenir les prédictions associés à ce modèle.

fig 7 : exemple api avec streamlit